

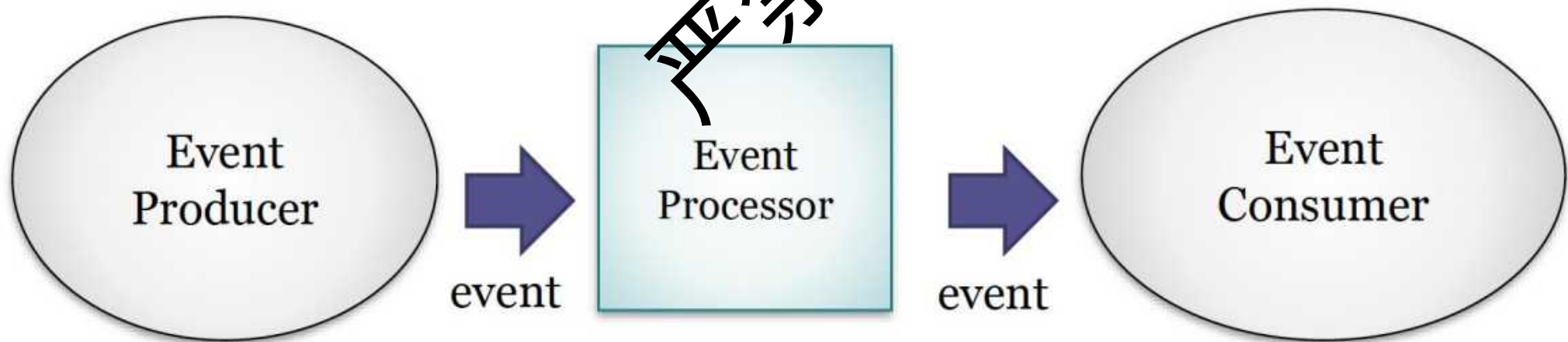
第9课 软件架构行为模式 part2

汇报人：北京交通大学软件学院

严禁复制

6 Event driven architecture

- 两种模式：
 - publish-subscribe
 - actor models



EDA的要素

> Event

已经发生的事件（例如客户下单了，不是请求，不是动作）

> Event消费者

数据库，应用系统，积分卡

> Event 生产者

事件的生成（传感器，或者其它系统）

> Event处理者

传输渠道
过滤和转换事件

严禁复制

EDA 约束

异步通信

one to many

- 生产者可以在任何时刻产生event
 - 消费者可以在任何时刻被通知event
 - 不要求立刻处理，也不等待请求结果返回（有时候不适用）
- 一个event可以被多个consumer接受到

严禁复制

EDA的优势和挑战

1

优势（相对于call return）

解耦合

生产者和消费者互相不依赖；

不阻塞

事件发布立即返回，不必等待处理结束；

灵活性

可以在process执行过程中发布event（不干扰流程）；

可测试性

分开测试生产者和消费者；

2

挑战

顺序不能保证

事件可能丢失，可能delay

一致性

难以debug

因为是异步执行

EDA的典型应用场景

- 事件处理可以帮助企业识别趋势和威胁，例如：
 - 资本市场的**交易员**希望对套利机会做出反应。
 - **情报分析人员**评估**卫星和传感器**数据流，以确定适当的进攻或防御行动。
 - **物流企业**利用**实时车辆遥测数据**更有效地管理车队。
 - **银行家**持续**追踪交易**以查找欺诈、洗钱或违反金融法规的行为。
 - **通信服务提供商**寻求最小化**网络故障**的平均修复时间（MTTR）。
 - **石油公司**根据实时数据动态确定**钻探范围**。
 - **汽车零部件**供应商提供用于**准时化生产**（JIT）的零部件。

6 Event driven architecture

- 典型应用

- Event processing networks

- “深度加工”和“智能分析”的网络结构，从大量、快速的事件中提取有价值的信息和模式。
 - 实时分析，感知趋势，自动响应

- Event-Stream-Processing (ESP)

- 对连续不断的事件流进行实时或近乎实时处理的技术和实践

- Complex-event-processing

- 是ESP的子集
 - 模式检测 (Pattern Detection): 这是 CEP 的核心
 - 事件关联 (Event Correlation): 将看似独立的事件联系起来，以发现它们之间的因果关系或相关性。
 - 事件聚合 (Event Aggregation): 对一段时间内或一组相关的事件进行统计计算，例如计数、求和、平均值等。

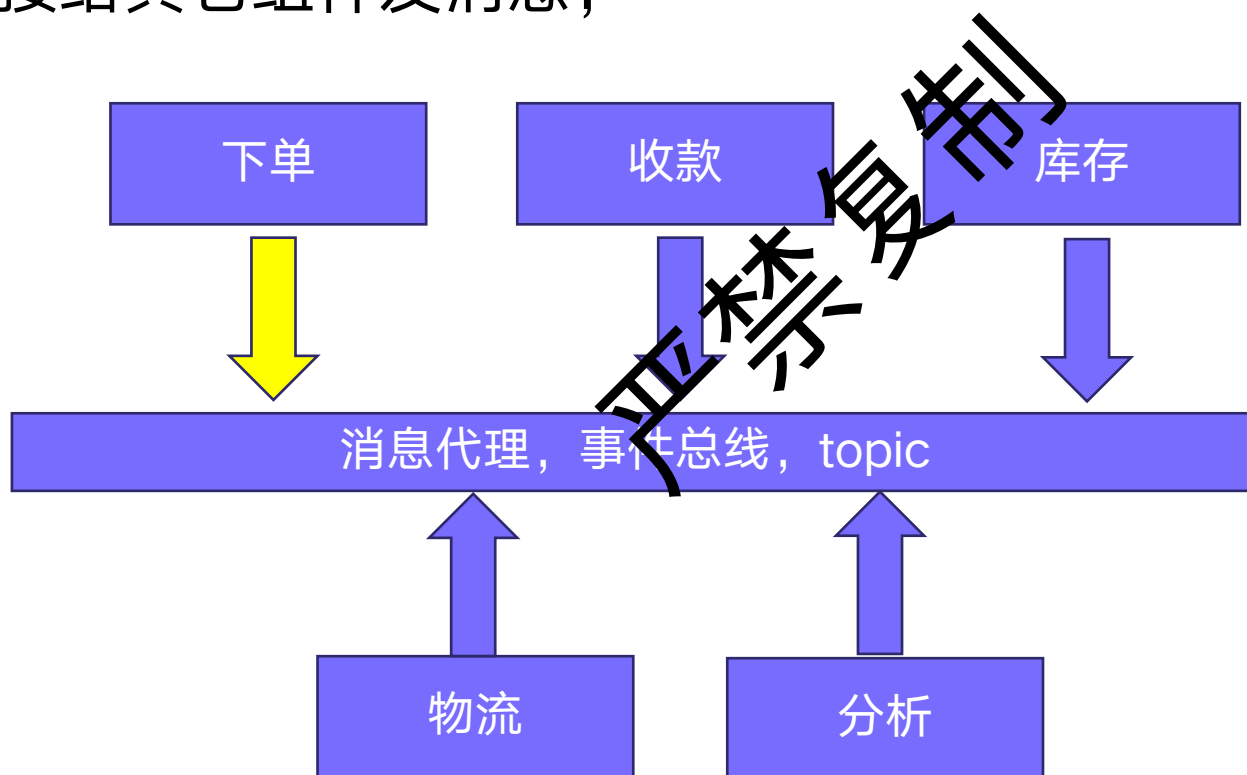
Event driven architecture

- 变化模式
 - Publish-subscribe
 - Actor models
- 关联模式
 - CQRS, Event sourcing

严禁复制

6-2 Publish-subscribe发布订阅模式

- 组件连接到事件总线并发出消息，同时接收其它组件发出的消息；
- 组件不直接给其它组件发消息；



发布订阅模式的要素

- 组件
 - 订阅通道的组件
- 发布端口
 - 注册用于发布消息
- 订阅端口
 - 注册用于接收某种消息
- 事件总线 (消息通道)
 - 向订阅者传递消息

```
# value_serializer 用于将发送的数据序列化为字节串
# 这里我们将字典转为 JSON 字符串, 再编码为 UTF-8 字节
producer = KafkaProducer(
    bootstrap_servers=[KAFKA_BROKER],
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    # 可选: 增加可靠性设置
    # acks='all', # 等待所有 ISR 确认
    # retries=3, # 重试次数
    # request_timeout_ms=60000 # 请求超时时间
)
```

```
# value_deserializer 用于将接收到的字节串反序列化为可用数据
# 这里我们将 UTF-8 字节解码为字符串, 再解析 JSON
consumer = KafkaConsumer(
    TOPIC_NAME, # 要订阅的主题 (可以是一个列表)
    bootstrap_servers=[KAFKA_BROKER],
    group_id=CONSUMER_GROUP_ID,
    auto_offset_reset='earliest', # 如果是新的 group_id, 从最早的消息开始消费
    # auto_offset_reset='latest', # 如果是新的 group_id, 从最新的消息开始消费
    enable_auto_commit=True, # 自动提交 offset (简单, 但可能丢失或重复处理消息)
    # enable_auto_commit=False, # 手动提交 offset (更可靠)
    value_deserializer=lambda v: json.loads(v.decode('utf-8'))
    # consumer_timeout_ms=1000 # 如果1秒内没收到消息, 迭代会超时抛出 StopIteration
)
```

发布订阅模式约束

> 订阅端口与发布端口的分离

一个组件可以同时拥有这两种端口

非直接通信

通常是异步通信

组件将传递消息的责任委托给通道

发布订阅模式

优势

提升沟通质量
高性能
容易debug
降低组件之间的耦合
消费者不依赖于消息发布者
消息发布者也不依赖于消费者

挑战

间接通讯有时候不如直接通讯更高效
需要保证成功，需要处理返回结果
复杂的实现
可能需要成熟的商业软件的支持（COTS）

严禁复制

思考题

- 题目一：是否适用EDA

- 用户付钱成功；
- 订单系统需要告诉物流系统“可以发货了”。
- 同时需要通知积分系统给用户加积分。

- 题目二：是否适用EDA

- 用户点击“去支付”按钮，
- 订单系统需要请求付款系统“扣款”。
- 订单系统需要知道扣款是成功还是失败；
- 订单系统跳转成功页面or提示用户支付失败。

严禁复制



模式6 EDA变体Actor Model

> 用于并发计算的场景中

主体是actors而不是objects
actors之间不共享数据
actors之间通过异步的消息传输

从1973年开始进行理论研究

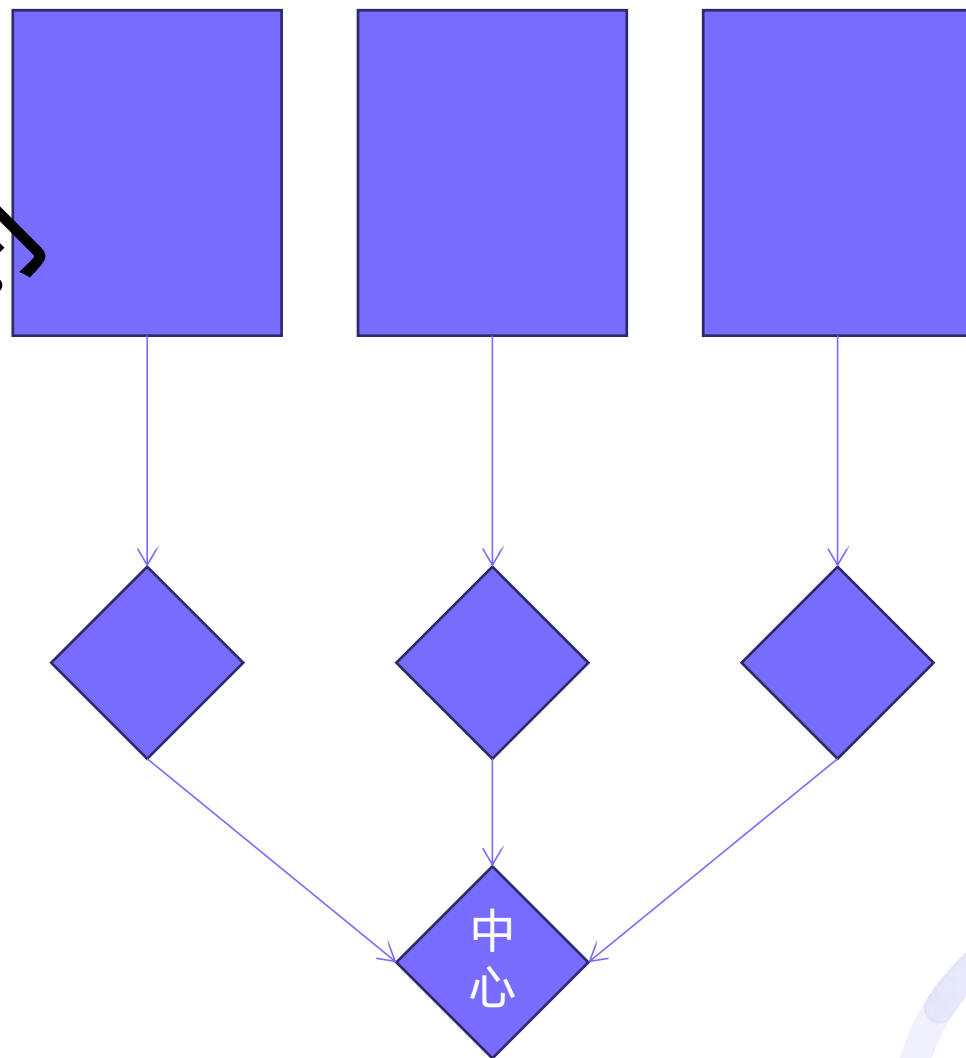
消息传递的保障，顺序，状态，粒度，
调试与可观测性

严禁复制

Actor Models要素

- actor: 具有计算能力的有状态的实体
 - 给其它actor发送消息
 - 按照顺序处理自己收到的消息
- 消息: 发送消息同步状态
- 地址: 每个actor都有自己的地址

严禁复制



Actor模型的约束

01

每个Actor只能:

- 给其它actor发送message (不可修改的)
- 可以创建新的actor
- 可以改变处理message的方式

02

actor之间是解耦合的

消息接收者不会依赖于消息的发送者

03

地址是局部变量, 不是全局变量

- 一个actor只能给自己知道的地址发送消息
- 地址是别人给的, 或者是自己创建的

04

并行

- 所有的动作都是并行执行的
- 没有共享的全局状态
- 消息是没有顺序的

严禁复制

Actor模型的优势和挑战

- 优势

高度并行化

透明性且可扩展

address是内在的，而不是外在的

可以扩展成全局的actor model

例如web 服务，多agent系统

- 挑战

消息发送（接收的消息可能会修改发送的消息）

actor的协调（先后顺序）

系统缺乏全局一致的状态（都是local的）

严禁复制

Actor模型

- 实现

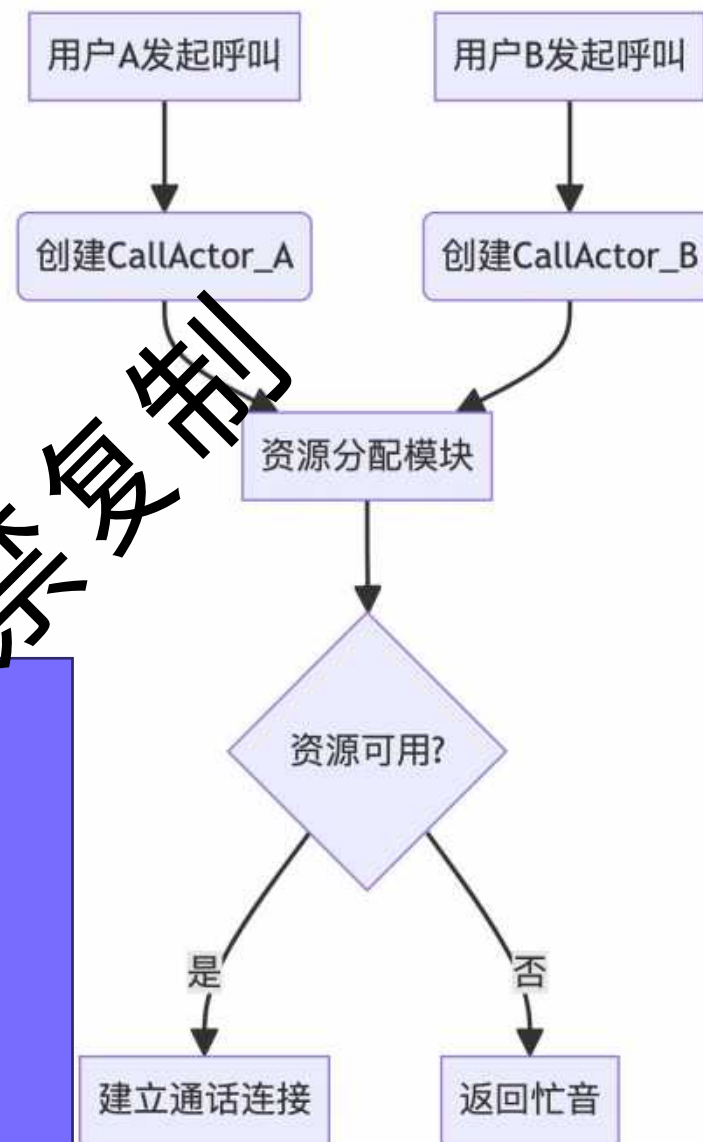
- Erlang (编程语言)
- Akka (框架)

- 系统

- 响应式系统
- 例如: Erisson, Twitter, Facebook

```
start_call(User) ->
  spawn(fun() ->
    Resource = resource_manager:allocate(), % 请求资源
    case Resource of
      {ok, Line} -> handle_call(User, Line); % 通话建立
      busy -> notify_user(User, busy)      % 返回忙音
    end
  end).
```

```
handle_call(User, Line) ->
  io:format("Call connected: ~p on line ~p~n", [User, Line]).
```



Actor模型-akka框架

- 概述

- 开源工具包，JVM

- 例子：

- 聊天系统：每个用户一个 Actor；
 - 物联网：每个设备一个 Actor；

ActorSystem	管理 Actor 的容器，提供线程池、调度器等基础设施。
ActorRef	Actor 的引用，用于发送消息（类似“地址”）。
Dispatcher	决定如何分配线程执行 Actor（默认基于线程池）。
Supervisor	监控Actor，定义崩溃恢复策略（重启/停止/上报）。
Akka Streams	基于 Reactive Streams 的流处理库（背压支持）。
Akka Cluster	构建分布式 Actor 系统（节点自动发现、负载均衡）。

```
// 定义 Actor
class Greeter extends Actor {
  def receive = {
    case "hello" => println("Hello back to you!")
    case _       => println("Unknown message")
  }
}
```

```
// 创建 ActorSystem 和 Actor
object Main extends App {
  val system = ActorSystem("MySystem")
  val greeter = system.actorOf(Props[Greeter], "greeter")

  greeter ! "hello" // 发送消息
  greeter ! "hi"    // 触发未知消息
}
```

Hello back to you!
Unknown message

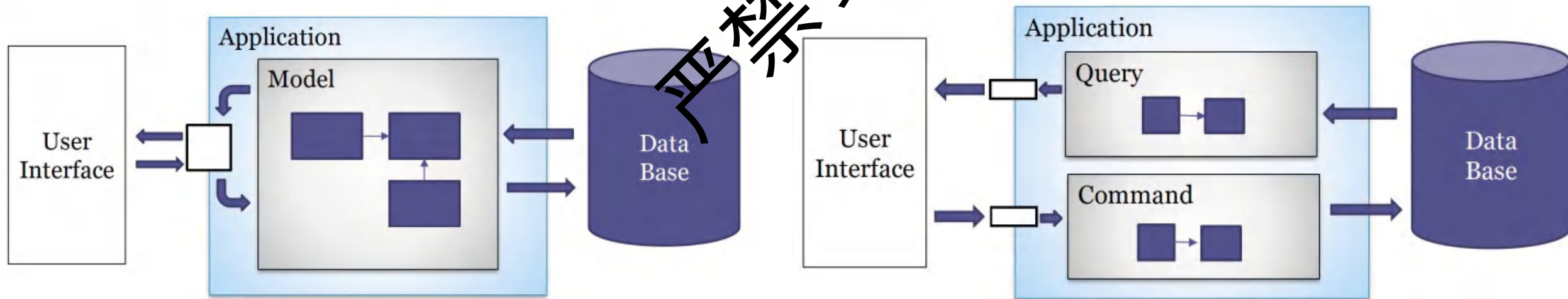
思考题

- 为了确保防空系统的有效性，需要对导弹拦截进行模拟和测试。请问在这个模拟系统中，是否适合使用actor model进行建模？
- A 适合
- B不适合



模式6-4 CQRS

- 命令和查询职责分离
 - 命令command: 发生改变（创建，删除，修改）
 - 查询query: 查询（获取信息）



CQRS适用场景

- 高性能、高并发读场景:

- 当应用的读操作远多于写操作，且读取的性能要求非常高时（例如，社交媒体信息流、电商商品展示页、新闻门户）。
- 优势: 读写操作的数据模型不同，存储技术不同

- 复杂的业务领域模型:

- 写模型可能包含复杂的业务规则和验证逻辑，而读模型可能只需要扁平化、及规范化的数据以便于展示。
- 优势: 写模型专注于处理业务逻辑和保证数据一致性，读模型专注于高效、便捷地提供数据。

- 与事件溯源 (Event Sourcing) 结合:

- 在 Event Sourcing 中，写命令被存储下来。读模型构建视图。
- 优势: 可以回溯到任何时间点的状态。

- 协作开发场景:

- 在大型项目中，不同的团队可能分别负责核心业务逻辑和数据展示/报表。
- 优势: 降低团队间的耦合度。

- 需要不同一致性模型的场景:

- 写操作通常需要强一致性，确保业务规则得到满足。而读操作只需要最终一致性，以换取更高的性能和可用性。
- 优势: CQRS 允许为读写两端选择不同的一致性策略。

CQRS的挑战

> 难以处理读写混合的操作：

例如stack的pop操作，既有读操作又有写操作；

> 实现复杂度比较高：

简单的CRUD应用不需要分离；

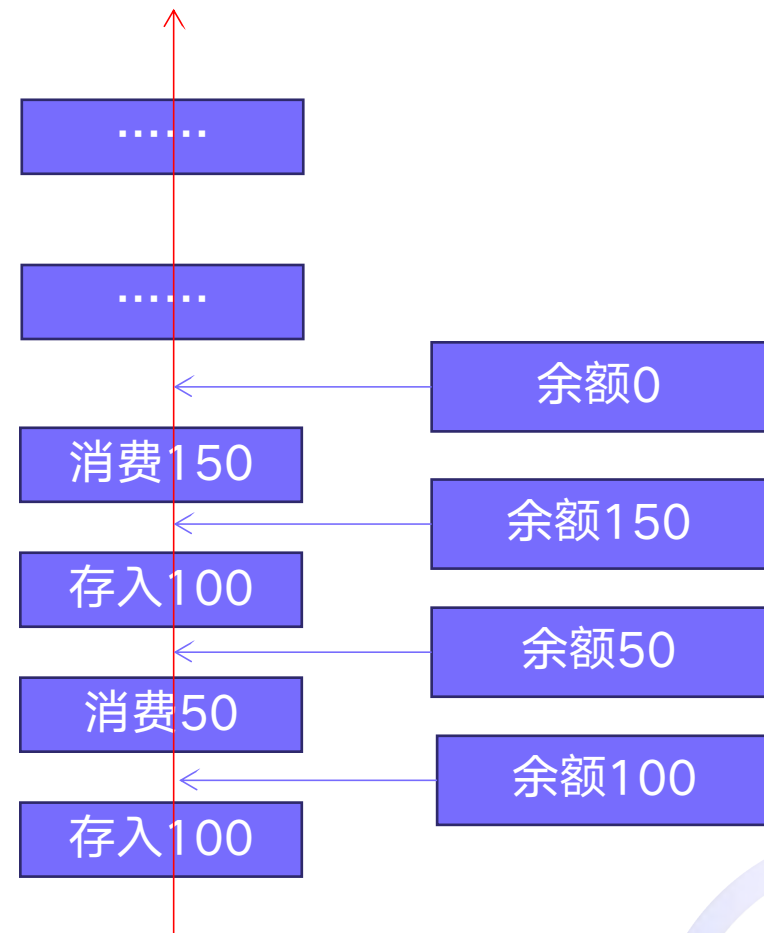
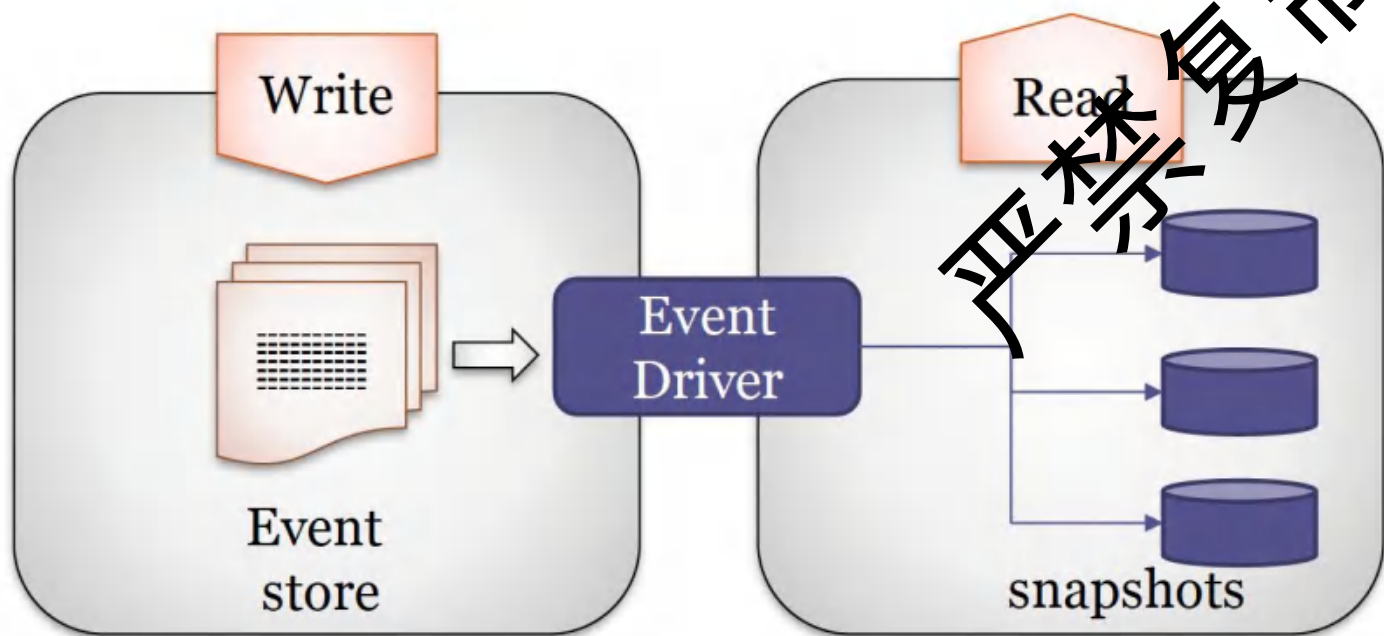
> 读写不同步的问题

有可能内容已经修改了，但是读到的是未修改的版本（库存）；

严禁复制

模式 6-5 Event Sourcing事件溯源

- 所有对系统状态的改变操作都当成event存起来
 - 每一个改变都存在event store之中；
 - 可以跟踪或者取消改变；



事件溯源

- 要素

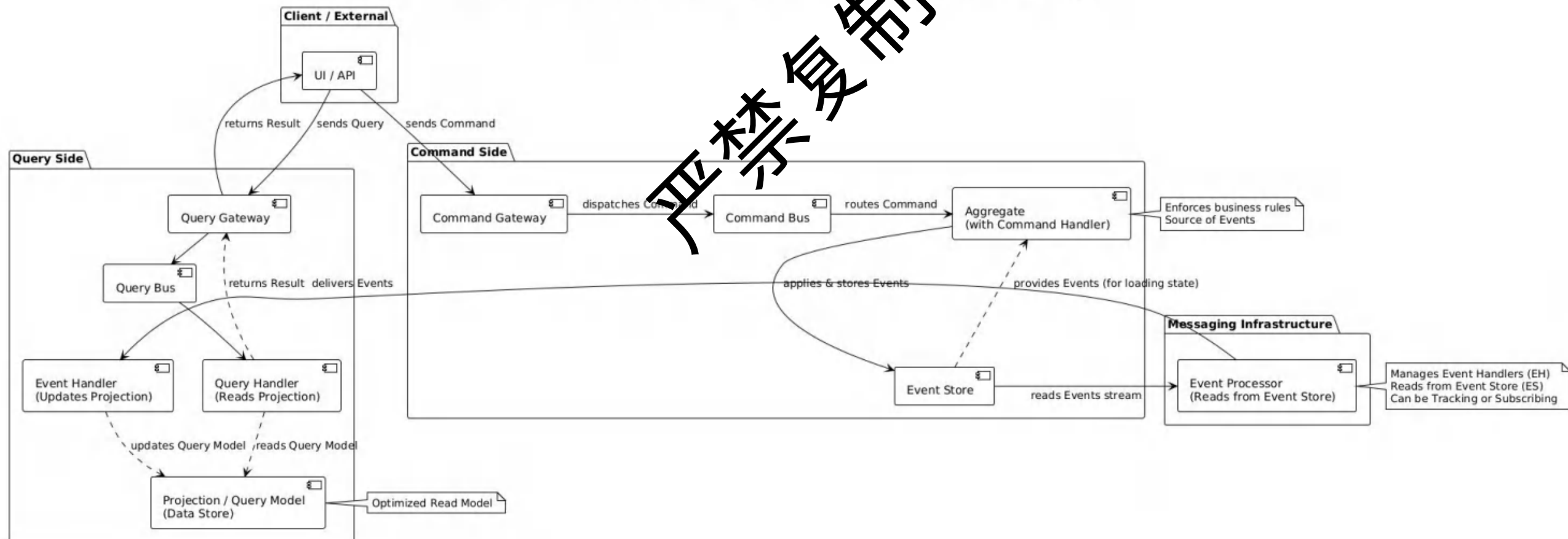
- 事件：过去已经发生的事情
- 事件存储：append only
- event driver：（事件筛选，事件解释）
- snapshots：事件聚合（每周，每小时，每分钟，……）

严禁复制

事件溯源的实现-axon框架

- Axon Framework 通过提供高级抽象、基础设施组件（如 Event Store）、性能优化（Snapshotting）、维护性支持（Upcasting）和强大的测试工具，显著降低了实施和维护事件溯源架构的复杂性，提高了开发效率和系统的健壮性。它使得开发者能够更容易地享受到事件溯源带来的好处（如完整的审计日志、时间旅行能力、灵活的查询模型构建等）

Axon Framework Core Components (CQRS & Event Sourcing)



事件溯源的优势和挑战

优势

高容错

状态变化可跟踪

重建状态：可以剔除错误状态

高可扩展：append only

严禁复制

挑战

开发模式的改变：不再需要

update……where

最终一致性：不是强一致性

软件升级：多版本的event

资源管理：存储所有的event需要大量资源

事件溯源的应用系统

- 数据库系统

- datomic

- 不可变数据 (Immutability): 最核心的特点。数据一旦写入, 就永远不会被修改或删除
 - 你可以查询数据库在过去任何一个时间点的状态
 - 原子事实: ID 、属性 、值 、事务ID (Transaction ID), 添加or撤回。
 - 完整的历史记录和审计: 由于数据永不删除, 这对于审计、调试、合规性至关重要。

- EventStore

- 原生支持事件流 (Event Streams): 它的核心数据结构就是事件流 (不是record)。
 - 强一致性保证: 对于单个事件流的写入操作, 提供强一致性保证。
 - 优化的追加写入和流读取:
 - 内置投影 (Projections): 生成新的事件流或构建读取模型 (状态聚合)
 - 时间旅行和审计:

时间溯源和事务日志的区别

特性/维度	关系型数据库事务日志 (Transaction Log)	Event Sourcing (事件溯源)
核心概念	数据库内部机制	应用层架构模式
主要目的	保证 ACID (持久性、原子性)、崩溃恢复、数据复制	捕获完整的业务状态变更历史，作为事实的单一来源 (Single Source of Truth)
抽象层次	低级别，以数据为中心 (如 SQL 语句、页面修改)	高级别，以业务/领域为中心 (如 OrderPlaced, CustomerMoved)
主要用户/消费者	<u>数据库管理系统 (DBMS)</u>	应用程序逻辑 (聚合、命令处理器、投影、订阅 重建聚合状态、构建查询模型 (Read Models / Projections)、 <u>审计、业务分析</u> 、系统集成)
数据保留期	通常是短期的，备份/ <u>检查点后可能被截断或归档</u>	通常是长期的或永久的，代表不可变的业务历史
与应用的关系	实现细节，通常对应用层隐藏	应用模型的核心部分，显式设计和使用
相似之处	都是顺序追加 (<u>Append-only</u>) 的记录变更	都是顺序追加 (<u>Append-only</u>) 的记录变更

第7类模式 高适配性的系统

这些架构模式旨在提高系统的灵活性、可扩展性和可维护性，**适应变化的需求!**

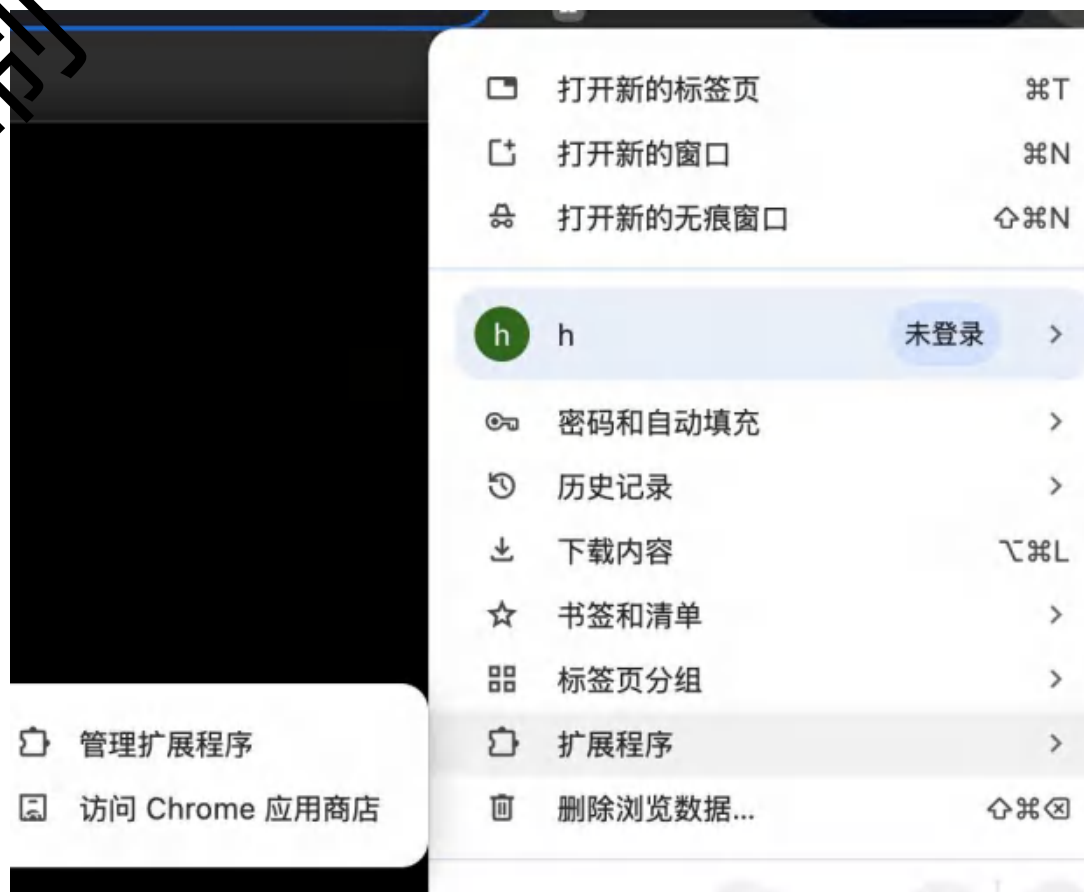
1. Plugins
2. Microkernel
3. Reflection
4. Interpreters and DSL
5. Mobile code
 - Code on demand
 - Remote evaluation
 - Mobile agents

严禁复制

7-01 Plugins

- 定义：一个系统允许他人通过增加plugin的方式实现新的功能；
- 要素：
 - 基础系统：
 - 基本的系统功能
 - plugins：
 - 可以动态添加或移除的组件
 - runtime engine：
 - 启动，初始化，执行，停止插件
- 典型案例：
 - 浏览器；
 - 集成开发环境；

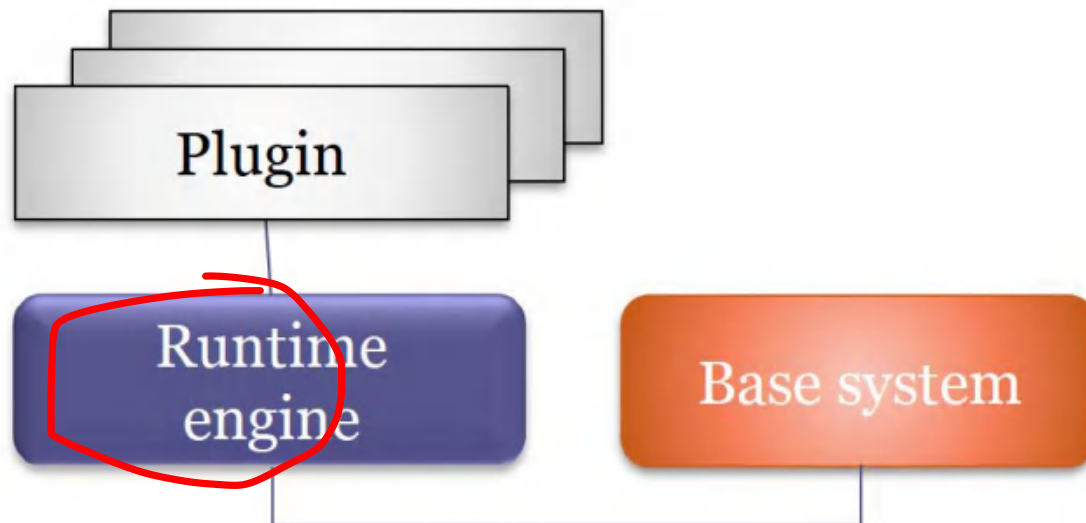
严禁复制



7-01 插件模式

- 约束

- runtime engine负责管理plugin,
- 系统可以增加或删除插件, 而不会影响系统整体;
- 插件之间可以有依赖关系
 - 插件必须明确声明自己依赖了谁, 以及自己提供的接口是什么



7-01 插件模式

- 优势:

1. 可扩展

1. 系统可以获得开发时未预见的功能

2. 可定制

1. 应用程序可以很小，后续再扩展

- 挑战

1. 一致性

1. 所有插件必须保持一致

2. 性能

1. 插件可能影响性能

3. 安全性

1. 插件可能包含安全漏洞

4. 插件的依赖管理

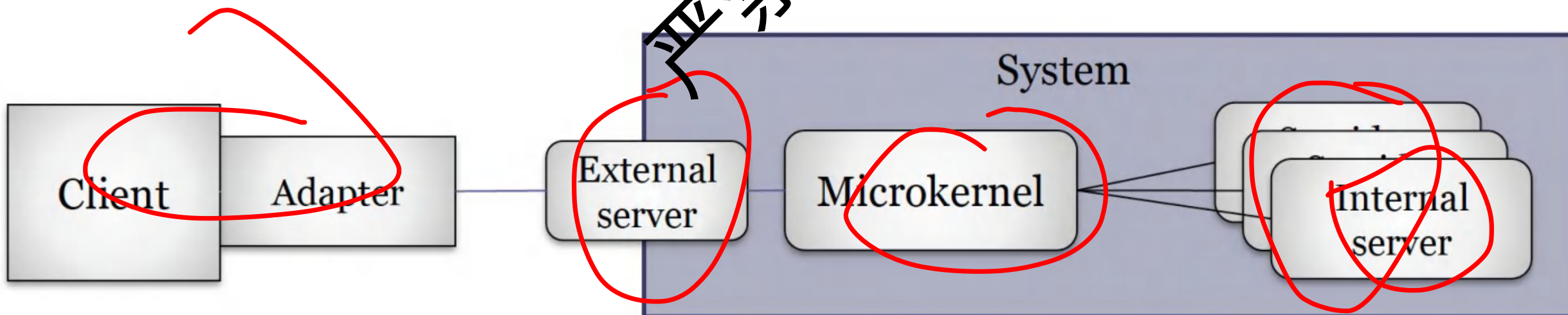
思考题

- 手机上的app是插件吗？
 - A 算，因为它是操作系统的一部分
 - B 不算，因为它是独立运行的

严禁复制

7-02 Micro kernel 微内核

- 在微内核中实现最小的功能；
- 内部服务实现额外的功能；
- 外部服务提供其它系统调用的接口；
- Client: 外部系统



7-02 微内核模式的约束

01

微内核只实现最小的功能

最基本，最核心

例如：进程调度，内存管理，进程间通讯

02

内部服务

在微内核之外运行的独立进程

例如：设备驱动程序，文件系统服务，网络协议

03

外部服务

向客户端提供接口

例如：创建文件，创建进程，创建图形界面

7-02 微内核

优势

1. 高可移植性
 1. 支持kernel就能支持整个系统
2. 高可扩展性
 1. 通过增加internal server的方式
3. 高安全性
 1. 核心部分被封装起来
4. 高可靠性
 1. 外部服务的错误不会影响微内核

挑战

- 性能
1. 微内核性能高但是容量有限
 2. 设计复杂度
 1. 定义微内核需要较强的抽象能力
 3. 单点失效
 1. 如果微内核失效了，整个系统都会崩溃

7-02 微内核的思考

- 微内核主要用于操作系统（区分内核空间，用户空间）
- 微内核架构的核心思想
 - 小核心、模块化、清晰接口、可扩展、故障隔离
- 对普通软件系统的启示
 - 识别内核：识别业务系统中最核心、最稳定、变化最少的部分
 - 例如用户认证、核心交易流程、基础数据模型；
 - 内部服务：业务系统也应将不同的业务能力设计成相对独立的模块或服务
 - 如订单管理、库存管理、支付处理、通知服务、报表生成等
 - 外部服务：这意味着模块/服务之间需要有稳定、定义良好的通信协议和接口
 - 如REST API、gRPC、消息队列的事件格式
 - 故障隔离：实现一定程度的故障隔离。
 - 如断路器、超时控制、重试机制、舱壁模式

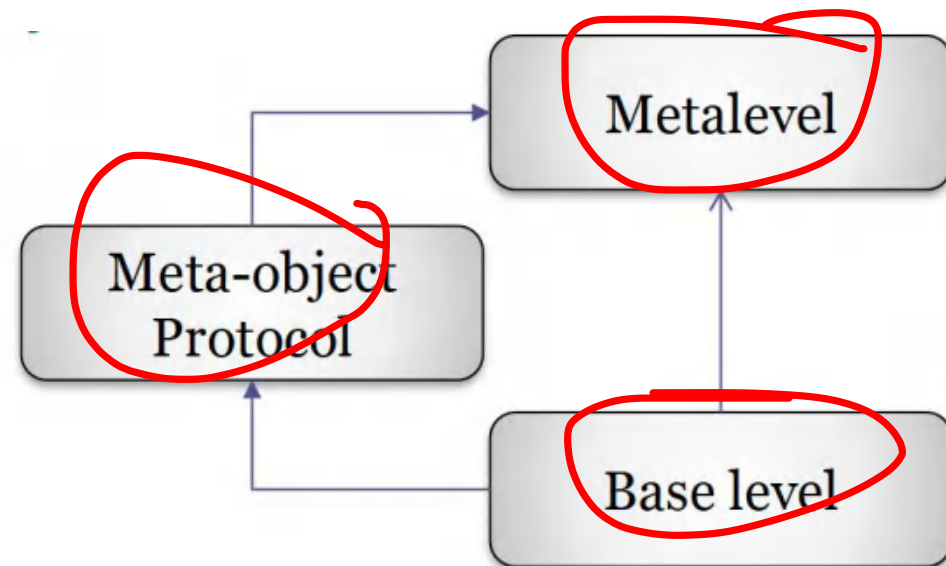
7-03 Reflection 反射模式

- 核心能力

- 反射是一种机制，它允许一个应用程序在运行时（而不是编译时）动态改变结构（增加字段）和行为（增加方法）
- 能够自我修改或自我审视的系统。程序能够了解自己内部的构造，并对其进行操作。

- 关键要素

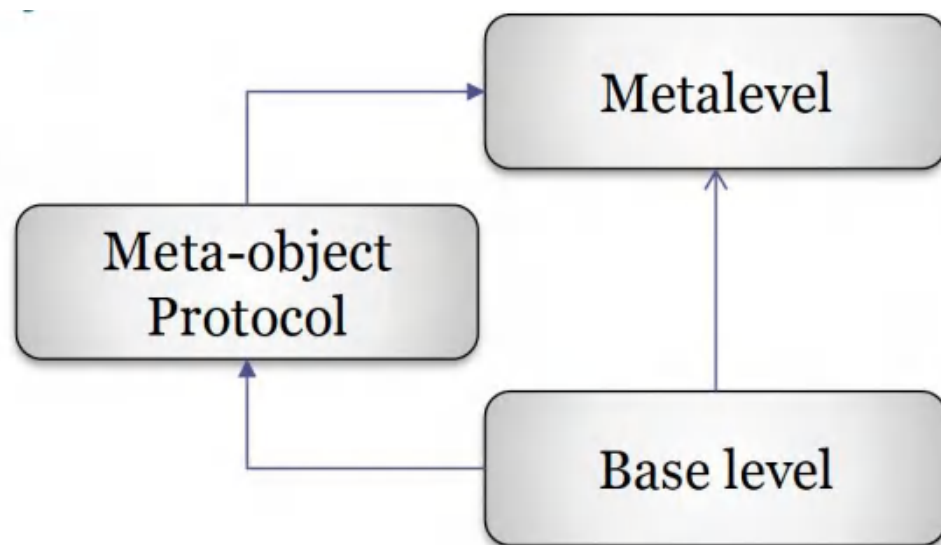
- 基础层 (Base level):
 - 这是应用程序的“常规”部分，业务代码、功能实现等。
- 元层面 (Metalevel):
 - 基础层代码的描述信息（元数据），比如类的定义、方法签名、字段类型等。
- 元对象协议 (Metaobject Protocol - MOP):
 - 元层面访问协议，例如：查询类信息、获取方法列表、调用私有方法、创建对象实例、读写字段值等。



7-03 反射模式

- 约束

- 基础层使用元层面来决定其行为
 - 例如一个log程序，如果发现是集合对象，就获取元素的tostring
 - for x in collection
 - log.info(x.toString())
- 在运行时，基础层可以使用元对象协议修改元层面
 - 系统根据配置文件决定采用何种策略
 - Console logger
 - file logger



7-03反射模式

优势

灵活性

适应运行时的条件变化

不必修改代码就能改变行为（如log）

挑战

实现

并不是所有语言都支持反射（java支持）

对于静态类型系统比较困难（不支持动态改变属性）

性能

反射比较消耗性能，接口是比较好的替代品

安全性

维护一致性比较困难

严禁复制

7-03 反射模式的应用场景

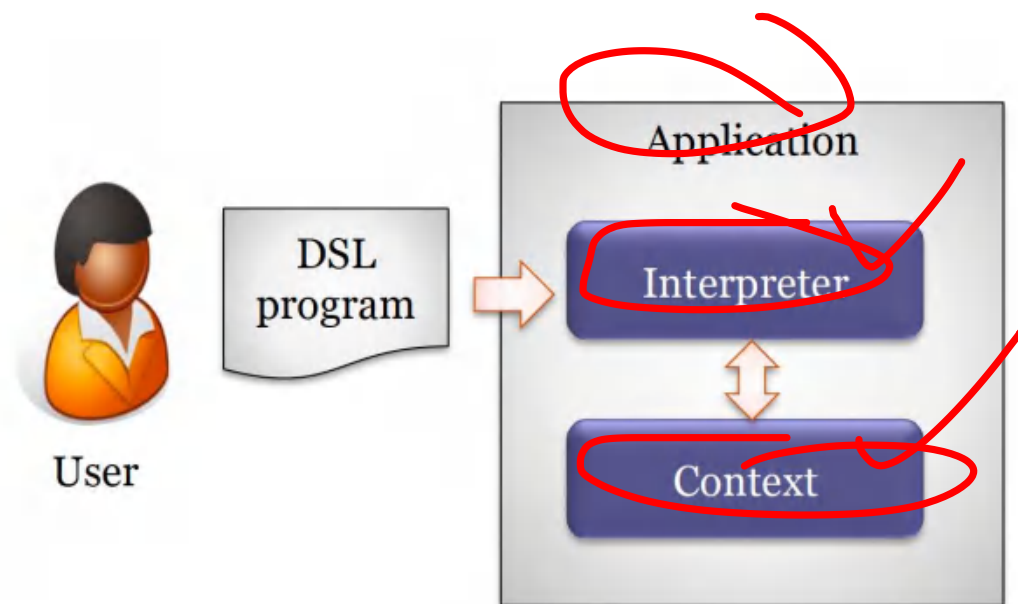
- 大多数动态语言支持反射
 - Scheme, Clos, Ruby, Python……
 - type(my_list), hasattr(s, 'append')
- 智能系统
 - 内省自身能力: 使用反射检查它当前加载了哪些 Skill .
 - 动态规划/决策: 基于任务需求和自身能力的了解, Agent 可以动态地选择并组合合适的技能序列来完成任务。
- 自我修正的代码
 - (eval `(defun ,new-func-name ,new-func-params ,new-func-body))

7-04 Interpreters and DSL解释器模式

- 当用户需要实现一个功能时，
- 不是直接编写对应的代码
- 而是设计一种特殊的语言（DSL）来描述功能
- 再让系统将DSL翻译成可执行的语言（SQL, JAVA, ……）

excel

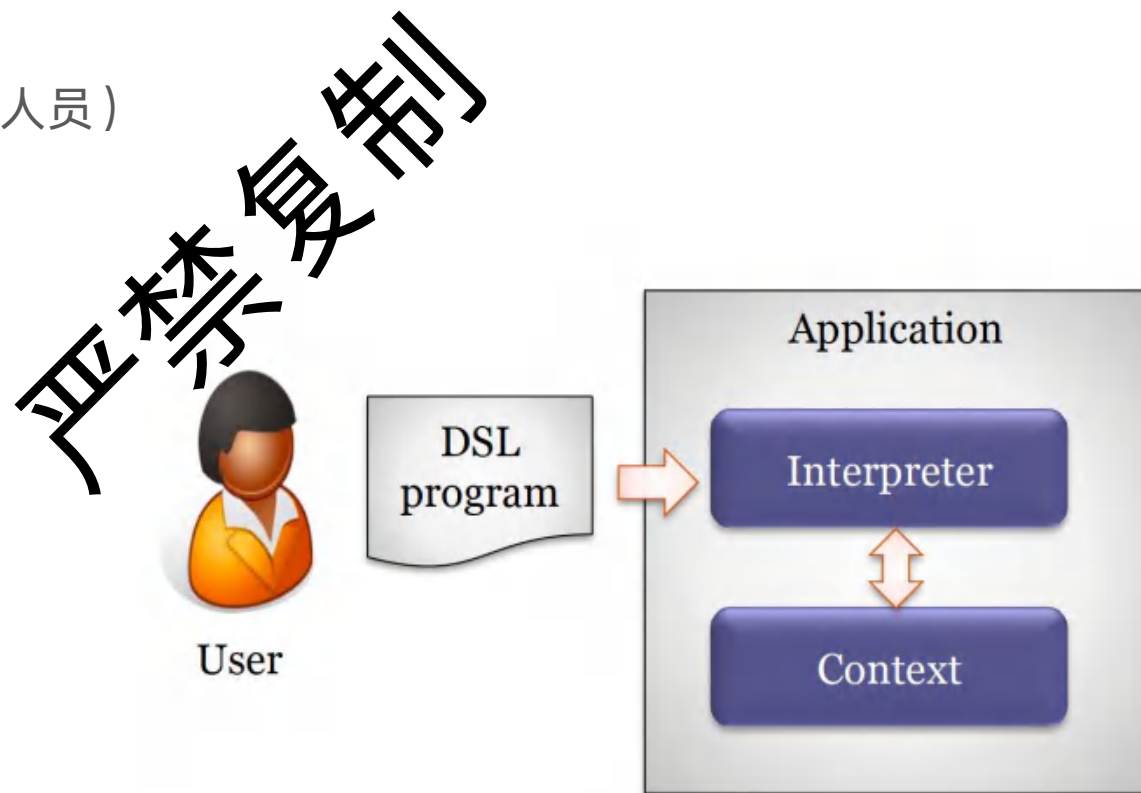
严禁复制



7-04 解释器模式

- 要素

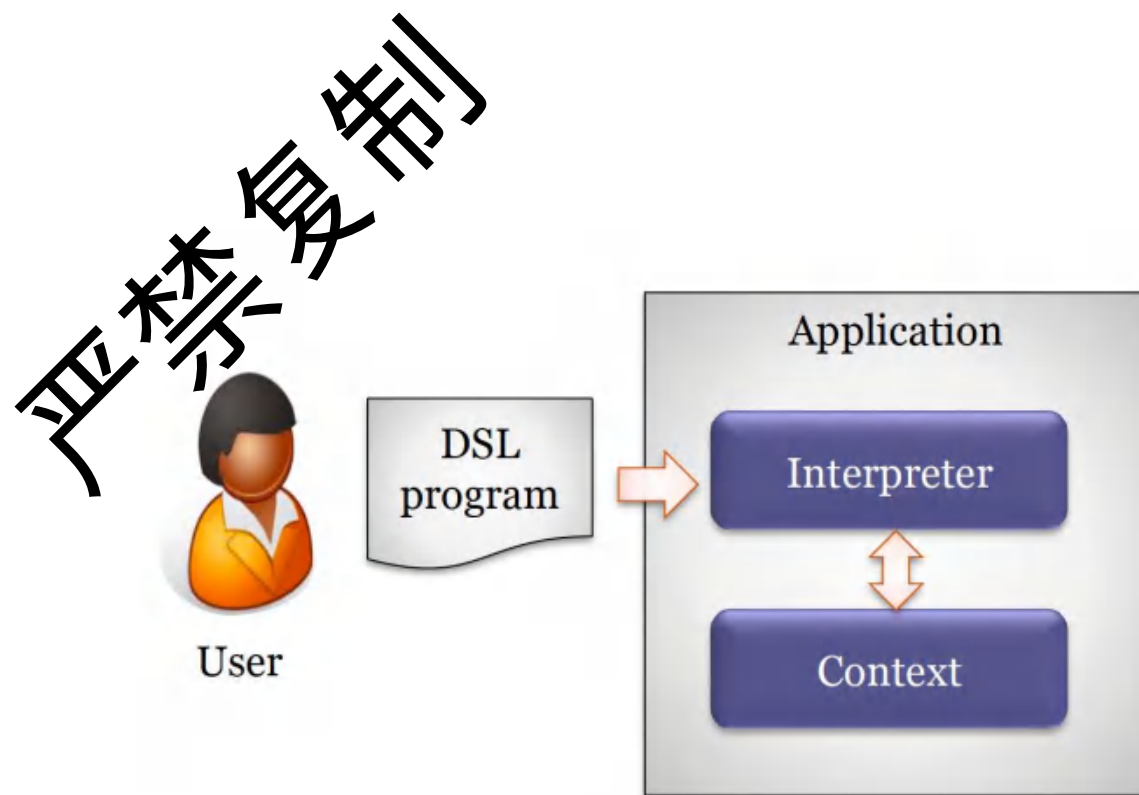
- 解释器：执行程序的模块
- 程序：使用DSL编写
 - 可以让最终用户编写（而不是研发人员）
- 上下文：程序执行的环境



7-04 解释器模式

- 约束

- 解释器可以在上下文中执行DSL程序
- 必须定义DSL
 - 语法（支持解析）
 - 语义（执行动作）



7-04 解释器模式

• 优势

- 灵活性：
 - 根据终端用户的需求修改系统
- 易用性
 - 终端用户可以根据需要调整系统
- 适配性
 - 可以支持未知的需求

• 挑战

- 设计DSL
- 实现的复杂性
- 分离上下文和解释器
- 性能
- 安全性：
 - DSL可能隐含漏洞

7-04 解释器模式

选择解释器模式：

- 通过特定的小语言定义计算逻辑、查询条件，且有比较明确的文法结构。
- 例如：报表计算字段、自定义过滤条件、简单脚本执行。

选择规则引擎：

- 将复杂的、多变的业务决策逻辑从核心代码中分离出来，让业务人员能够方便管理。
- 例如：信贷审批、风险评估、促销活动配置、 workflows 路由。

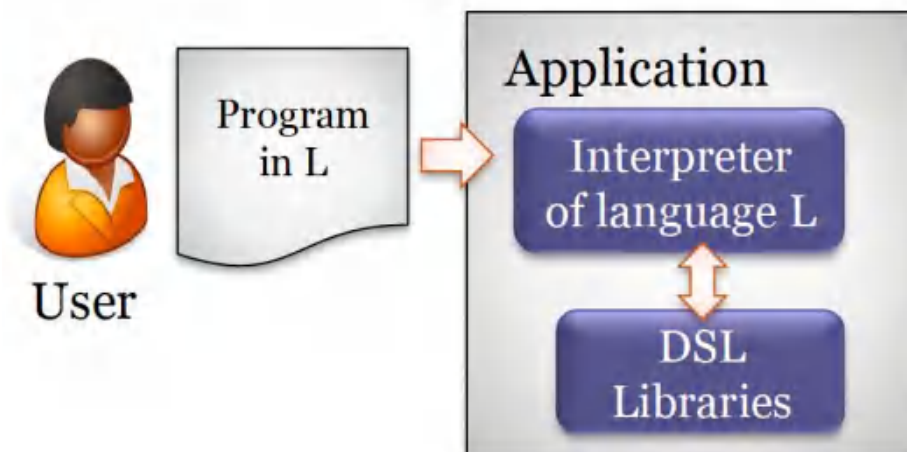
7-04a 解释器变体

- 内嵌的小语言

严禁复制

7-04a内嵌的小语言

- 嵌入通用宿主语言的领域特定语言 (Embedded DSLs)
 - 在某些语言（如 Haskell、Ruby、Scala 等）中是一种流行的方法。
- 优点：
 - 复用宿主语言的语法
 - 可以访问宿主语言的库和集成开发环境 (IDE)
- 挑战：
 - DSL 与宿主语言之间的分离度（或：界限问题）
 - 最终用户可能获得过多的表达能力（或：表达能力过强）



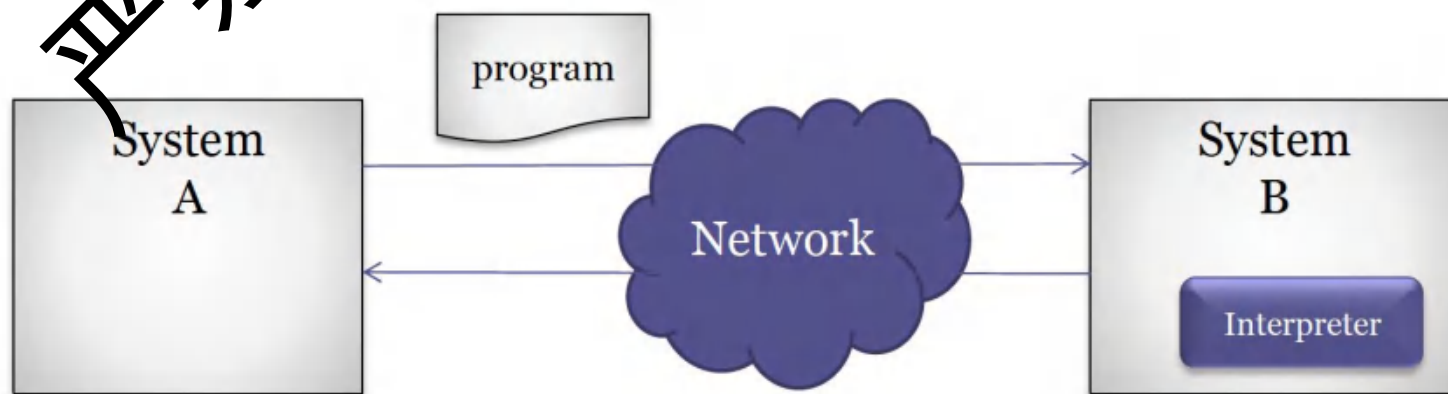
7-05 Mobile Code 移动代码

- 在机器间传输的代码

- 指从一台机器传输到另一台的代码。
- 系统 A 发送一个程序，由系统 B 来运行。
- 系统 B 必须包含该程序所使用的编写语言的解释器

- 例：

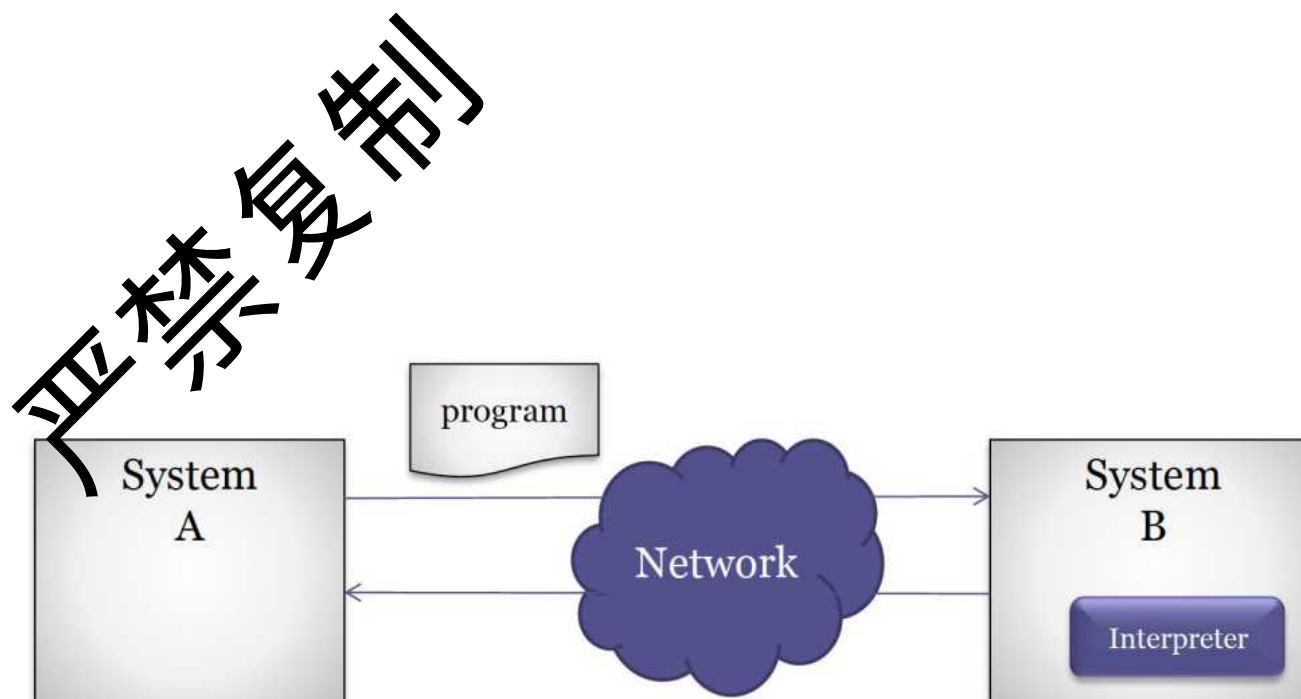
- Web 浏览器中的 JavaScript。



7-05 移动代码模式

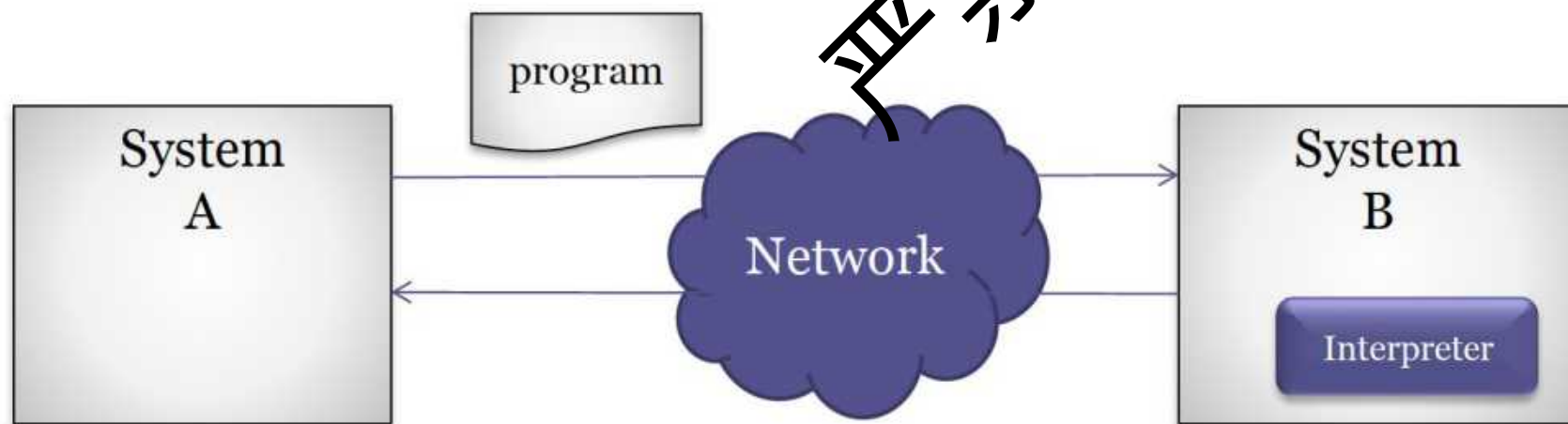
- 要素

- 解释器：执行被传输的程序
- 程序：被传输的代码
- 网络：



7-05 移动代码模式

- 约束
 - 程序必须在接收端执行
 - 网络协议支持代码的传输



7-05 移动代码

- 优势
 - 灵活适配新环境
 - 并行执行
- 挑战
 - 实现的复杂性
 - 安全性（可能危害接收方）

严禁复制

7-05移动代码的3种变体

01

按需代码

客户端 拉取 代码，在 本地 执行（JavaScript）。

02

远程求值

客户端 推送 代码/请求，在 远程 执行，通常返回结果。

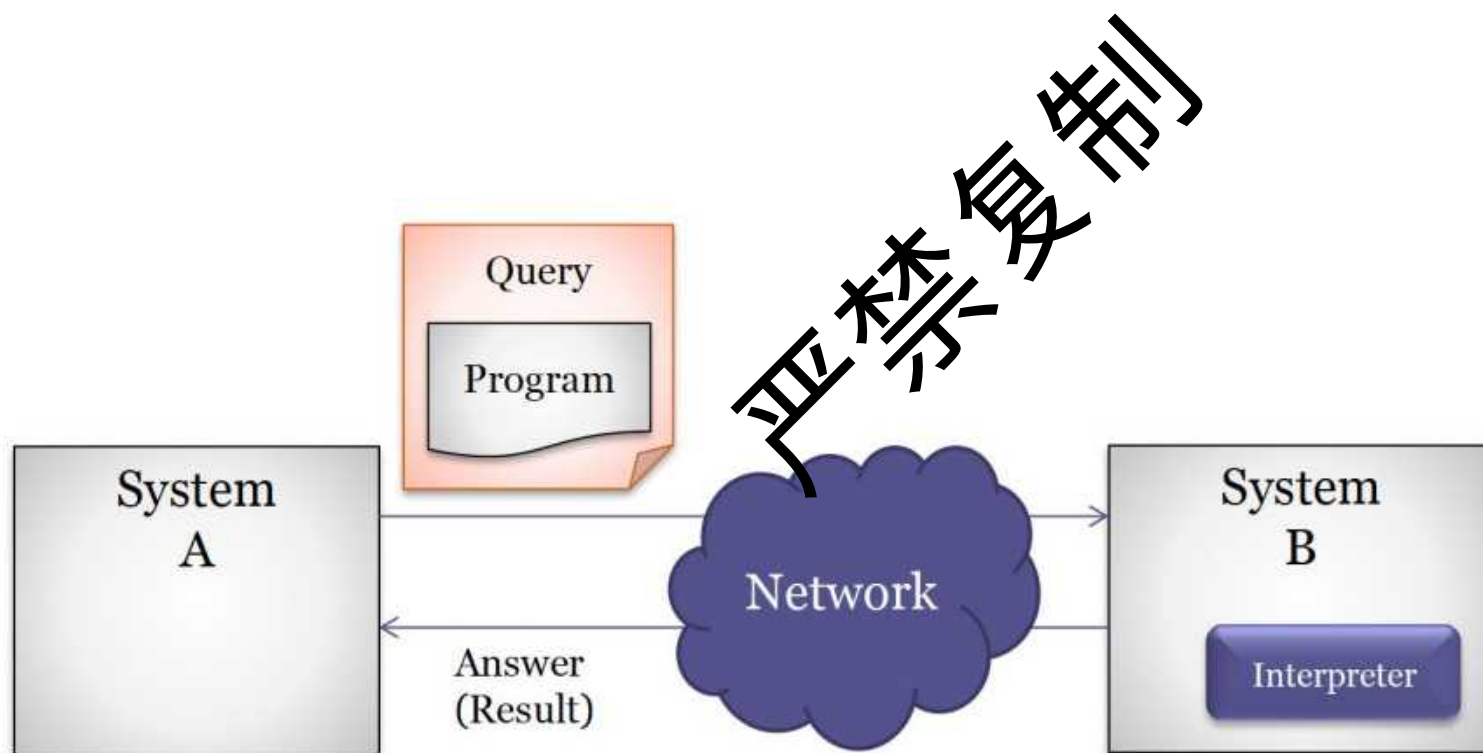
03

移动代理

完整的程序（代码+状态）在多个主机间 自主移动，并在 多个 远程 主机上依次执行。

7-05b 远程求值

- 系统A发送程序到系统B，系统B运行后把结果返回



7-05b远程求值

- 要素 (Elements)
 - 发送方 (Sender): 发起包含程序或代码的查询/请求。
 - 接收方 (Receiver): 执行程序并返回结果。
- 约束条件 (Constraints)
 - 程序在接收方执行。
 - 接收方必须能够理解和执行接收到的代码。
 - 接收方需要有相应程序语言的解释器 (Interpreter), 或者能够直接执行机器码 (Machine Code)。
 - 需要网络协议来传输程序 (或请求) 以及执行后的结果。
- 优势:
 - 能够利用远程的CPU 内存, ……
- 挑战
 - 不信任的代码, 病毒

7-05b 远程执行

- 例子

- BOINC (Berkeley Open Infrastructure for Network Computing):

- 它提供了一个通用的框架，利用志愿者计算的模式。
 - BOINC 平台负责处理任务分发、结果回收、用户积分、跨平台兼容性等通用问题。
 - 它本质上就是一个大规模实施远程求值（通过志愿者计算）的基础设施。

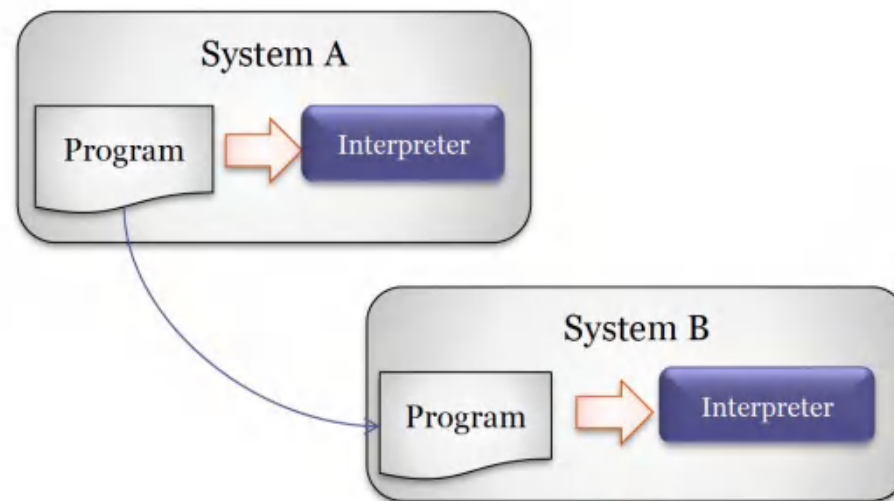
- Folding@HOME:

- 研究蛋白质折叠、误折、聚合及其与疾病（如阿尔茨海默病、癌症等）的关系。

- Predictor@Home: 进行蛋白质结构预测。

7-05c 移动代理模式

- 移动代理：
 - 能够在网络中的不同计算机（主机/节点）之间自主地迁移（移动）。
 - (Code) (Data) (Execution State) 打包移动新环境，从中断的地方继续执行其任务。
- 关键特征 (Key Characteristics):
 - 迁移性 (Mobility): 代理的核心能力是跨网络节点移动。
 - 状态保持 (State Preservation):
 - 代理会携带其内部状态（如变量值、执行点）。
 - 自主性 (Autonomy):
 - 代理通常被设计为能够独立运作，
 - 可以根据环境变化自主决定何时以及向何处迁移。



7-05c 移动代理模式

- 信息检索：在大型分布式数据库或信息库中查找特定信息。
 - 应用方式: 代理在本地执行搜索、过滤和聚合操作。这避免了传输大量原始数据。
- 点对点系统：在 P2P 网络中搜索文件、共享资源或执行分布式任务。
 - 应用方式: 代理可以在对等节点之间移动，代表用户搜索文件，或者协调节点间的协作任务。
- 电信：网络管理、服务配置、性能监控。
 - 应用方式: 代理在设备本地执行诊断、收集详细性能数据、动态配置服务或执行管理任务，减少了中央管理站点的负担和网络流量。
- 远程控制 and 监控：监控和管理地理上分散的设备、传感器或系统。
 - 应用方式: 部署移动代理到远程设备上。代理可以在本地持续监控设备状态。

7-05c 移动代理模式

- 优势

- 减少网络流量
- 代码随行执行（状态+数据）
- 隐式并行性
- 对网络故障的容错性
- 概念上的简单性
- 支持创建移动代理系统
- 涌现行为（多个agent出现复杂的行为）
- 适应环境变化和反应式学习

- 挑战

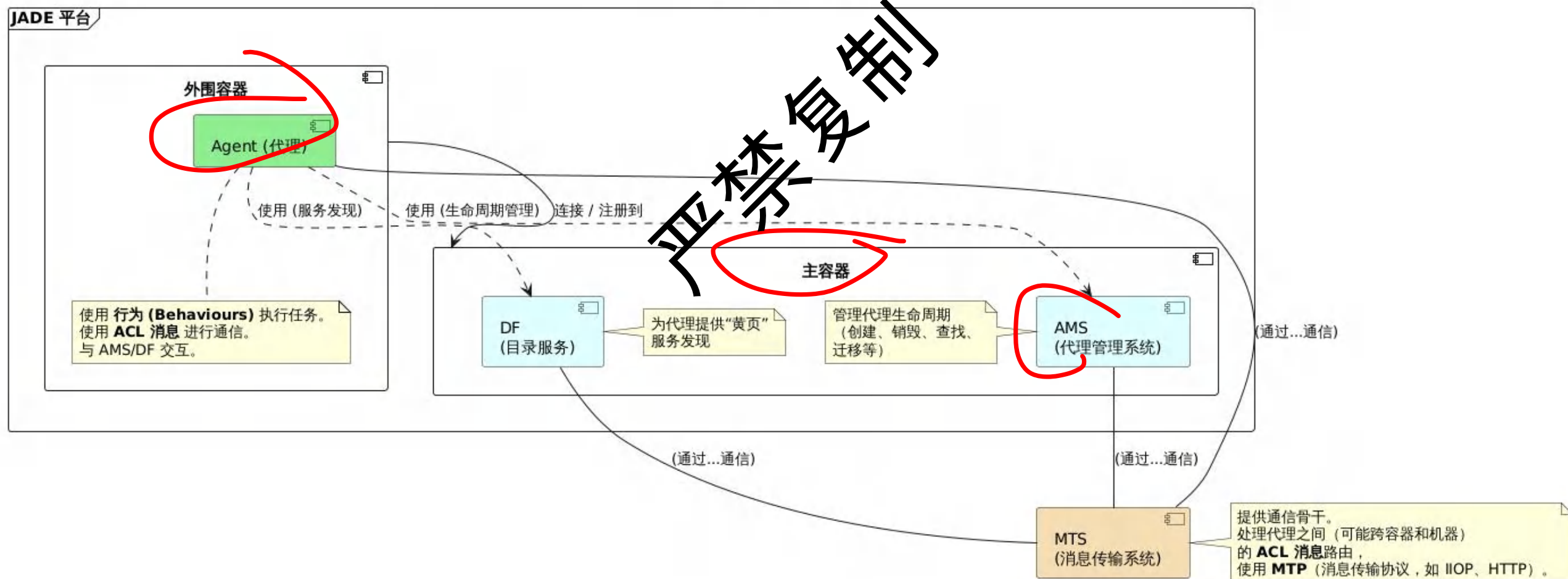
- 安全问题和恶意代码

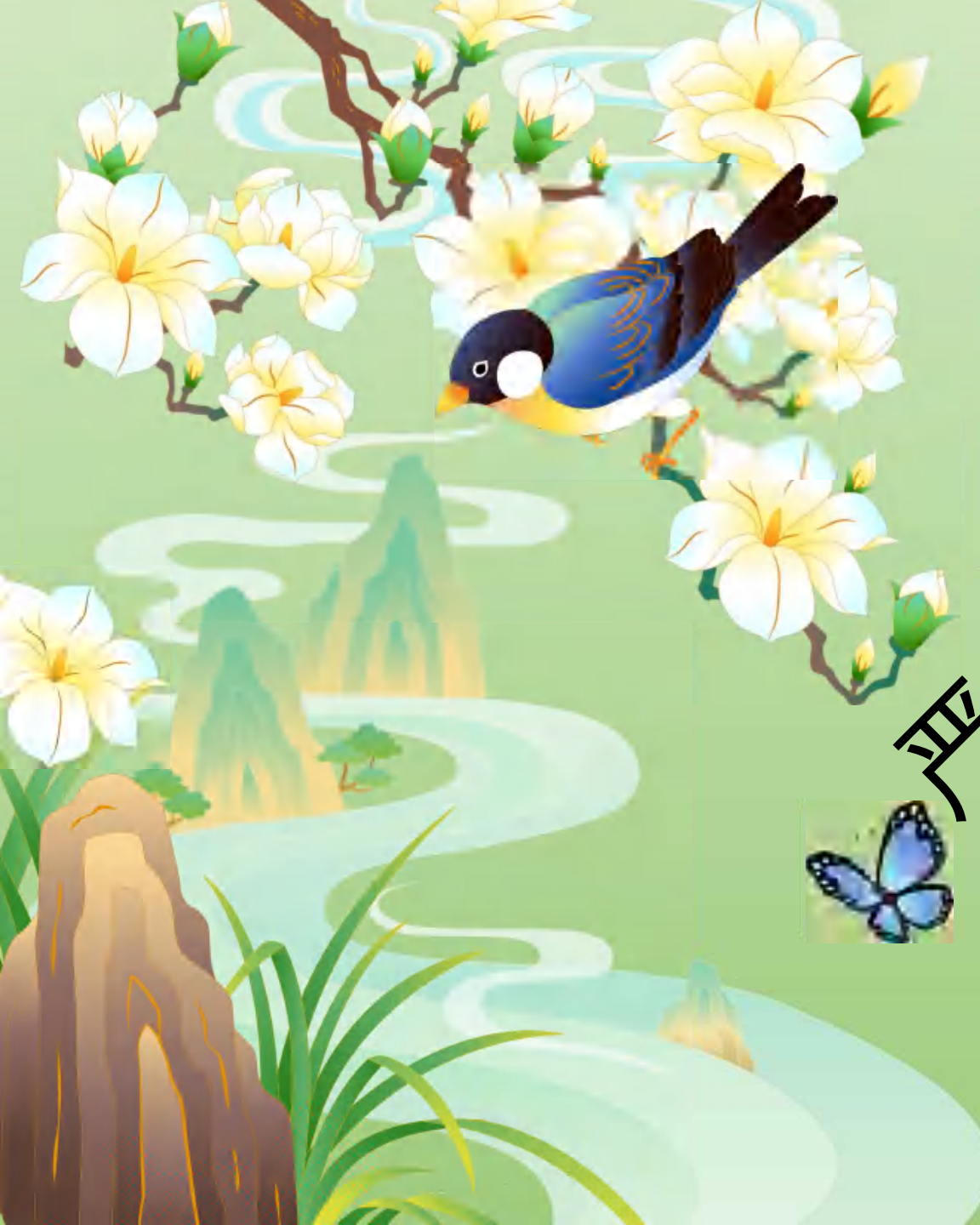
严禁复制

7-05c移动代理模式-JADE

- JADE 是一个用 Java 语言编写的多智能体系统开发框架

JADE 核心机制组件图





严禁复制

感谢观看

汇报人：北京交通大学软件学院